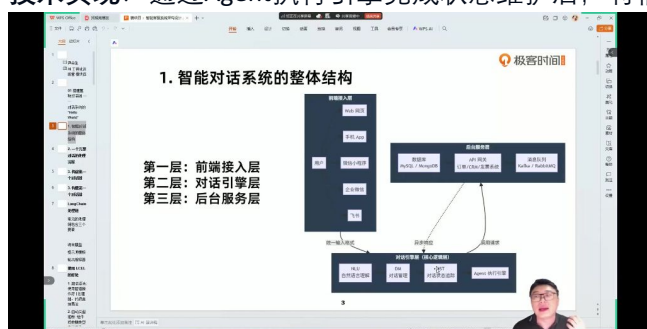


一、智能客服系统设计 00:16

1. 智能对话系统 02:35

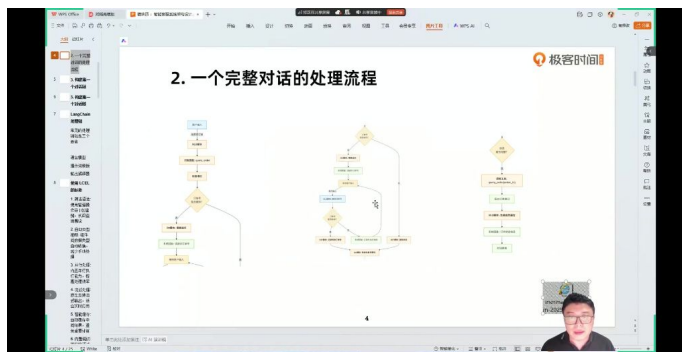
1) 智能对话系统的整体结构 05:26

- 智能对话系统顶层设计 05:28
 - 三层架构划分：系统分为前端接入层、对话引擎层（核心逻辑层）和后台服务层，不能仅靠大语言模型单独完成
 - 框架迁移趋势：未来企业需要支持OpenAI和Anthropic两套标准对话接口，如DeepSeek已实现双标准支持
- 前端接入层：定义与功能 05:50
 - 终端类型：支持Web网页、手机App、微信小程序、企业微信、飞书等多渠道接入
 - 核心功能：
 - 维护统一输入格式（类似OpenAI的标准化协议）
 - 处理异步响应调用请求
 - 提供良好的交互环境
- 对话引擎层：NLU自然语言理解 08:00
 - 处理流程：
 - 多模态输入预处理（语音转文字、图片识别等）
 - 意图识别（如"查询订单"）
 - 关键信息提取（如订单号）
 - 技术演进：从使用de scope微调转向Hugging Face的PEFT库实现
- 对话引擎层：对话管理 09:24
 - 决策机制：
 - 判断继续提问（如缺失订单号时反问用户）
 - 决定是否调用工具（如查询API）
 - 框架变更：从RAG框架转向LangChain框架实现
- 对话引擎层：对话状态追踪 09:44
 - 核心功能：
 - 支持跨会话记忆（第二天继续未完成对话）
 - 技术实现：通过Agent执行引擎完成状态维护后，将信息传递至后台服务层

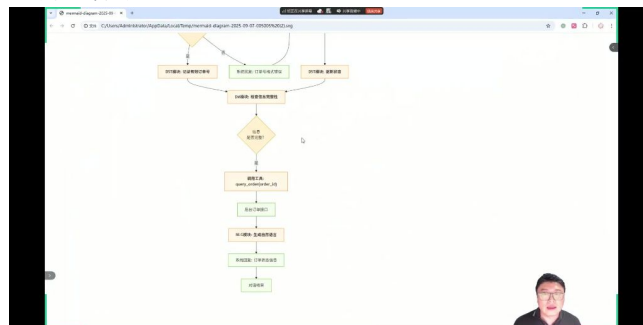


- 后台服务组成：
 - 数据服务：MySQL/MongoDB
 - 业务系统：订单/CRM/发票系统
 - 消息中间件：Kafka/RabbitMQ
 - API网关

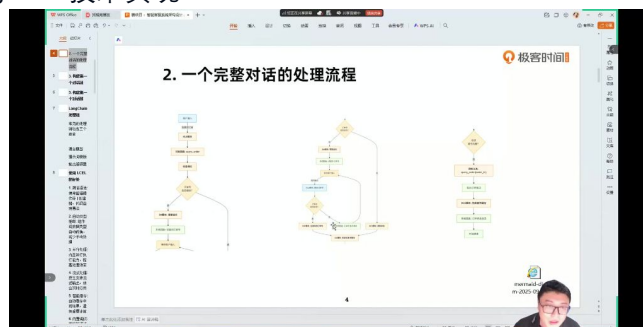
2) 智能客服系统设计流程 11:06



- 三层拆分：将完整对话系统拆分为NLU（自然语言理解）、DST（对话状态跟踪）、DM（对话管理）三个模块
- 模块协作：NLU负责意图识别 → DST更新对话状态 → DM检查信息完整性并决定下一步动作
- 订单查询业务流程 11:20



- 核心检查点：
 - 槽位填充：必须验证订单号等关键信息是否完整
 - 状态保持：支持用户中断后继续未完成流程
 - 工具调用：最终需调用后台接口完成查询
- Agent核心能力 13:38
 - 三大能力架构：
 - 大模型交互：通过提示词工程实现有效对话
 - 工具调用：对接业务系统执行具体操作
 - 记忆管理：保存对话上下文和中间结果
- LangChain技术实现



- 关键组件：
 - PromptTemplate：定义标准化指令模板（如"你是一个客服..."）
 - ChatModel：封装OpenAI/通义千问等大模型接口
 - OutputParser：将输出转换为字符串/JSON等标准格式
- 流程优化策略
 - LCEL六大优势：

- **管道操作符**: 简化代码结构
- **类型转换**: 自动处理数据类型
- **并行执行**: 提升处理效率
- **流式输出**: 支持实时响应
- **结果缓存**: 避免重复计算
- **内置记忆**: 自动保存对话状态

3) 实战构建流程 15:32

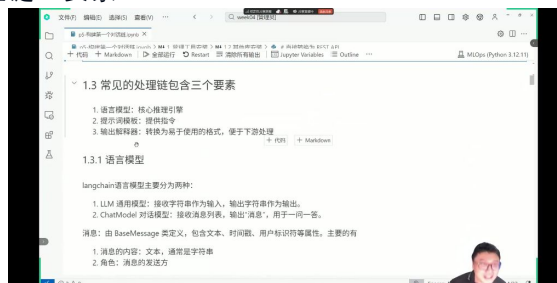
- **最小可行系统设计**
 - **简化假设**:
 - 用户意图明确为订单查询
 - 输入包含完整订单号
 - 工具调用100%成功
 - **核心链路**: 意图识别 → 槽位提取 → 工具调用 → 结果返回
- **异常处理机制**
 - **典型场景**:
 - **缺失槽位**: 通过DM模块发起追问
 - **格式错误**: DST模块验证并提示重输
 - **中断恢复**: 利用记忆功能保存对话进度

4) 构建第一个对话链 16:26

- **构建第一个对话链 19:30**
 - **构建第一个对话链**
 - **LangChain安装 19:57**



- **pip安装**: pip install langchain
- **conda安装**: conda install -c conda-forge langchain
- **版本说明**: 当前为1.0 alpha版本, 即将发布正式版
- **其他库安装 21:01**
 - **LangServe**: pip install "langserve[all]" 提供REST API功能
 - **LangSmith**: pip install -U langsmith 用于观测平台
 - **LangGraph**: 高级工作流抽象, 适合复杂流程
 - **DeepSeek集成**: pip install langchain-deepseek 支持国产大模型
- **处理链三要素**



- **语言模型**:

- LLM通用模型：字符串输入输出
- ChatModel对话模型：消息列表输入输出
- 提示词模板: 定义给模型的指令（如"你是一个客服..."）
- 输出解释器: 将输出转为标准格式（字符串/JSON等）

■ 链式处理机制



- 处理流程:
 - 用户请求输入
 - 提示词模板处理
 - 大模型推理
 - 输出解释器格式化
- 组合方式: 各组件通过"|"符号连接形成处理链

■ LCEL优势



- 语法简化: 使用管道操作符减少代码量
- 自动类型转换: 减少手动处理
- 并行处理: 内置并行执行能力
- 流式输出: 支持实时输出处理
- 智能缓存: 避免重复计算
- 调试支持: 更好的开发体验

■ 生产环境建议 24:39

- 环境变量管理: 使用python-dotenv管理密钥
- 接口标准化: 国内模型通常兼容OpenAI接口标准
- 社区扩展: 安装langchain-community集成更多外部资源

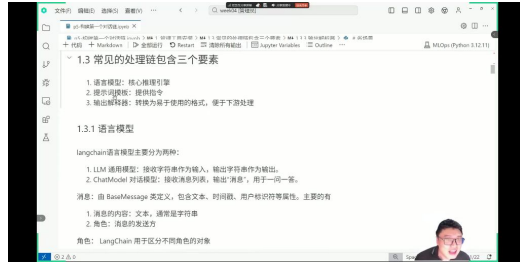
■ 开发注意事项

- 版本兼容性: 注意LangChain与各扩展库的版本匹配
- 模型切换: 不同模型可能需要调整URL和参数
- 性能优化: 复杂流程建议使用LangGraph进行封装

○ 常见的处理链 26:22

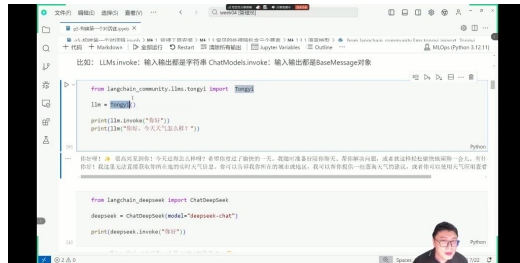
■ 语言模型

- 语言模型分类 30:27



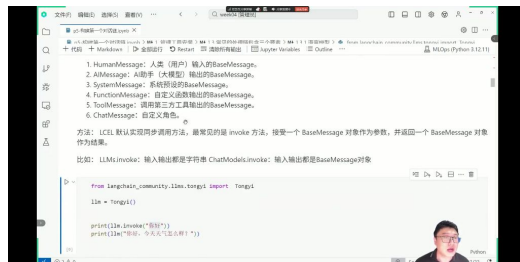
- LLM通用模型:
 - 输入输出: 接收字符串作为输入, 输出字符串
 - 特点: 最简单的未封装模型
 - 示例: Tongyi()直接调用
- ChatModel对话模型:
 - 输入输出: 接收消息列表, 输出BaseMessage对象
 - 特点: 封装了问答交互功能
 - 示例: ChatDeepSeek()调用

● 消息结构



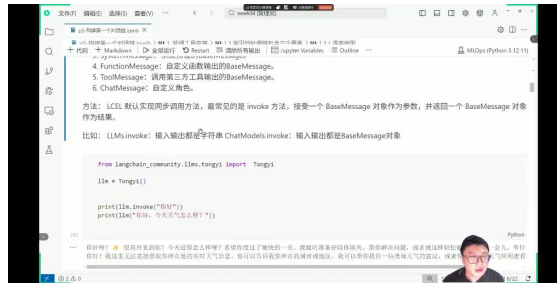
- BaseMessage类:
 - 包含属性: 文本内容、时间戳、用户标识符等
 - 核心要素:
 - 内容: 文本字符串
 - 角色: 消息发送方标识
- 角色类型:
 - HumanMessage: 用户输入
 - AIMessage: AI助手输出
 - SystemMessage: 系统预设
 - FunctionMessage: 自定义函数输出
 - ToolMessage: 第三方工具输出
 - ChatMessage: 自定义角色

● 调用方法



- LLM调用:
 - 方法: invoke()或直接调用
 - 特点: 输入输出均为字符串
 - 示例:

■ 使用区别



- **LLM特点:**
 - 简单直接，但定制性差
 - 适合基础对话场景
- **ChatModel特点:**
 - 自动封装字符串为消息列表
 - 支持更复杂的交互功能
 - 适合需要角色区分的场景

■ 实际应用



- **环境配置:**
 - 需要设置API密钥环境变量
 - 默认使用最新模型版本
- **调用方式:**
 - 同步调用: invoke方法
 - 异步调用: ainvoke方法 (未展示但存在)
- **响应处理:**
 - 包含内容文本和元数据
 - 示例响应结构:

■ 对话链构建基础

● 语言模型类型



- **LLM通用模型:** 接收字符串作为输入，输出字符串作为输出，如 llm.invoke("你好")
- **ChatModel对话模型:** 接收消息列表，输出BaseMessage对象，支持一问一答的交互模式
- **BaseMessage类型:**
 - HumanMessage: 用户输入
 - AIMessage: AI助手输出
 - SystemMessage: 系统预设消息

- FunctionMessage: 自定义函数输出
- ToolMessage: 第三方工具输出
- ChatMessage: 自定义角色消息

● 模型调用方法



○ invoke方法：

- 同步调用方法，LLM模型输入输出都是字符串
- ChatModel输入输出都是BaseMessage对象
- 示例：`deepseek = ChatDeepSeek(model="deepseek-chat")`
- 示例：`print(deepseek.invoke("你好"))`输出BaseMessage对象

● 处理链三要素



○ 核心组件：

- 语言模型：核心推理引擎
- 提示词模板：提供指令模板
- 输出解释器：转换输出格式

○ 工程实现：

- 支持LLM和ChatModel两种模型
- 消息传递使用BaseMessage对象封装
- 包含消息内容和角色属性

■ 提示词模板

● 基本模板



○ 定义：用于生成提示词的模板，如`{name}`、你好！我是你的朋友小爱同学

○ 变量规则：

- 由字母、数字和下划线组成
- 必须以字母开头

○ 创建方法：

○ 输出解析器 46:34

■ 核心功能

- **格式转换:** 将大模型原始输出转换为下游应用易用格式, 包括JSON、XML、列表等格式, 并能将ChatMessage转换为纯字符串
- **内容过滤:** 可消除消息以外的额外内容, 如自定义函数返回的附加信息
- **类型转换:** 将LangChain处理后的数据转换为Python基础数据类型 (列表、字典、元组等)
- **数据验证:** 验证输出是否符合预期格式要求
- **错误重试:** 解析失败时具备自动重试机制
- 常见解析器类型
 - JSON解析器
 - **实现方式:** 使用PydanticOutputParser定义目标类型结构
 - **工作流程:**
 - 定义期望的JSON结构类型
 - 创建对应解析器实例
 - 将大模型输出解析为规范化的JSON格式
 - **应用场景:** 需要结构化数据交换的API调用、数据存储等
 - 列表解析器
 - **基本功能:** 将逗号分隔的字符串自动解析为列表
 - 示例: 输入"苹果,香蕉,橙子" → 输出["苹果","香蕉","橙子"]
 - **自定义配置:** 可指定其他分隔符进行解析
 - **实现原理:** 通过CommaSeparatedListOutputParser实现自动分割
 - 结构化解析器
 - **特殊功能:** 可解析为预定义的结构体/类实例
 - **扩展性:** 支持用户手动定义解析逻辑和输出结构
 - **应用优势:** 适用于需要强类型检查的复杂业务场景
- 解析器链式调用
 - **组件构成:** 提示词模板 + 大模型 + 输出解析器
 - **连接符号:** 使用管道符|实现组件串联
 - 语法示例: prompt | llm | parser
 - **执行流程:**
 - 提示词输出作为大模型输入
 - 大模型输出作为解析器输入
 - 最终返回解析后的结构化结果
 - **效率优势:** 相比手动分步调用, 代码更简洁且执行流程更清晰
- 应用实例
 - 基础示例

```

1 - 创建水果列表生成链
2 fruit_chain = prompt | llm | parser
3 result = fruit_chain.invoke({"category": "水果"})
4 - 输出示例: ["苹果", "香蕉", "橙子", "葡萄", "西瓜"]
1 - 批量处理示例
2 categories = ["水果", "编程语言", "运动项目", "乐器"]
3 for category in categories:
4     print(chain.invoke({"category": category}))
5 - 可批量生成不同类别的列表数据
1 - 购物清单生成器
2
3 - **业务逻辑**: 根据营养需求参数生成结构化购物清单
4 - **参数示例**: 体重、卡路里需求、目标体重等
5 - **输出结构**: 包含早、午、晚餐食材的JSON格式清单
6 - **扩展应用**: 可适配各类清单生成场景 (旅行、会议等)
7 - 购物清单示例

```



```
8
9 - 输出解析器
10
11
12 - 
13
14 - **列表解析器**：使用`CommaSeparatedListOutputParser`将逗号分隔的字符串解析为列
表，例如"苹果，香蕉，橙子"解析为["苹果","香蕉","橙子"]
15 - **结构化解析器**：通过`StructuredOutputParser`和`ResponseSchema`定义输出结构，如
包含答案和来源两个字段的JSON格式输出
16 - **手动解析**：可以直接调用解析器实例进行手动解析，适用于需要自定义处理的情况
17
18 - 对话链构建
19
20
21 - 
22
23 - **链式组件**：由提示词模板、大语言模型和输出解析器三个核心组件组成
24 - **模板变量**：`input_variables`定义必需输入的变量（如餐型），`partial_variables`定义
固定不变的参数
25 - **链式调用**：使用管道操作符`|`连接组件形成链，如`shopping_prompt | llm | parser`
26 - **多餐型处理**：通过遍历["早餐","午餐","晚餐"]列表，分别调用链生成不同餐型的购
物清单
27
```

28 - 多场景扩展

29

30

31 -

32

- 33 - ****场景参数化****: 通过增加`category`和`item\_type`两个变量实现多场景适配
- 34 - ****通用链设计****: 保持解析器和格式指令不变, 仅修改提示词模板中的场景描述
- 35 - ****场景示例****: 包含早餐食材、办公用品、旅行必需品、健身器材、学习工具等多样化场景

36

37 - LCEL优势

38

39

40 -

41

- 42 - ****语法简洁****: 使用`|`管道操作符创建链, 代码直观易读
- 43 - ****类型推断****: 组件间数据类型自动转换, 减少手动处理
- 44 - ****高效处理****: 支持并行执行和流式输出, 适合实时应用
- 45 - ****智能特性****: 自动缓存中间结果, 内置调试和监控能力
- 46 - ****扩展性****: 支持自定义组件和统一的Runnable接口

47

48 - FastAPI封装

49

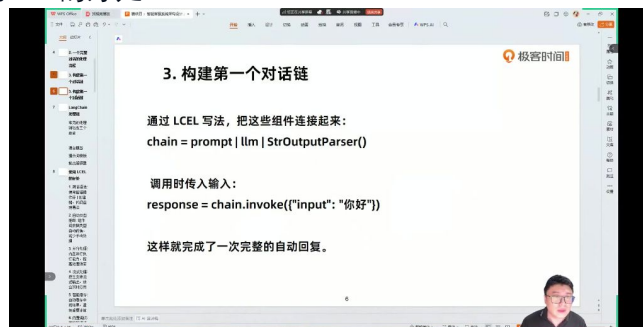
50

```

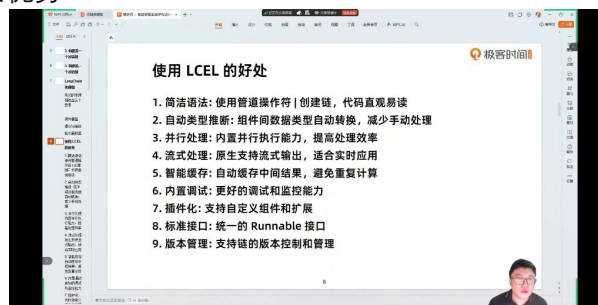
51 - 
52
53 - **服务架构**: 包含主应用、数据模型、对话链、会话管理和路由模块
54 - **会话管理**: 通过'session_id'维护对话历史, 实现多轮对话上下文
55 - **接口设计**: POST请求处理消息, 返回包含回复和会话ID的JSON响应
56 - **链式集成**: 将LangChain对话链封装为类方法, 通过'self'调用链组件

```

● 使用LCEL的好处 01:08:55



- 基本语法: 通过管道操作符连接组件, 格式为 `chain = prompt | llm | StrOutputParser()`
- 调用方式: 使用 `chain.invoke({"input": "你好"})` 完成一次完整的自动回复
- 主要优势



- 简洁语法: 使用管道操作符|创建链, 代码直观易读
- 自动类型推断: 组件间数据类型自动转换, 减少手动处理
- 并行处理: 内置并行执行能力, 提高处理效率
- 流式处理: 原生支持流式输出, 适合实时应用
- 智能缓存: 自动缓存中间结果, 避免重复计算
- 内置调试: 提供更好的调试和监控能力
- 插件化: 支持自定义组件和扩展
- 标准接口: 统一的Runnable接口规范

-

- ## ○ 模板引擎对比

- ## ■ Jinja2特点:

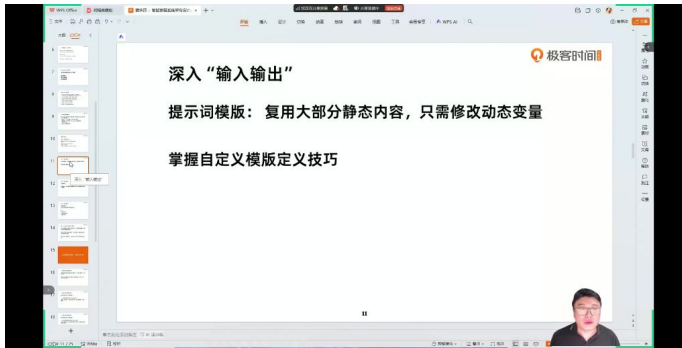
- ### ■ 应用建议:

- ### ○ 网络服务封装

- [illegible]

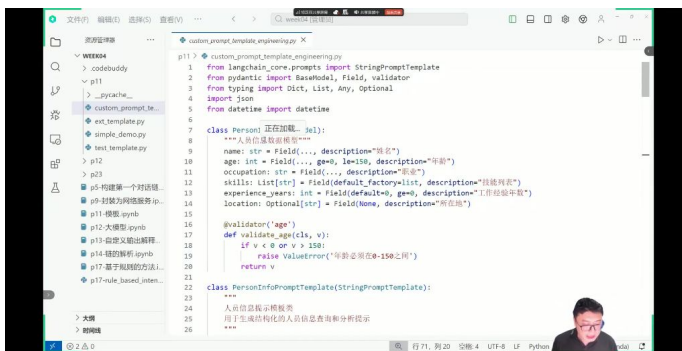
- ## 二、深入输入输出 01:19:13

1. 模板设计技巧 01:20:06



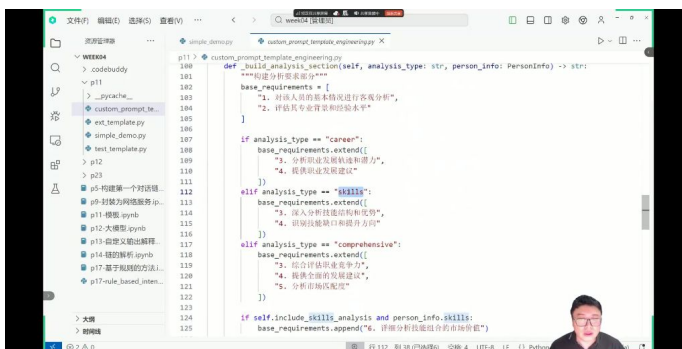
- 静态嵌套变量：基础模板采用静态内容嵌套动态变量的方式，适用于简单场景
- 动态模板扩展：企业级需求需要掌握动态模板技术，实现更灵活的提示词组合
- 工程化实现：通过Python类封装模板逻辑，支持参数验证和功能扩展

1) 核心自定义模板



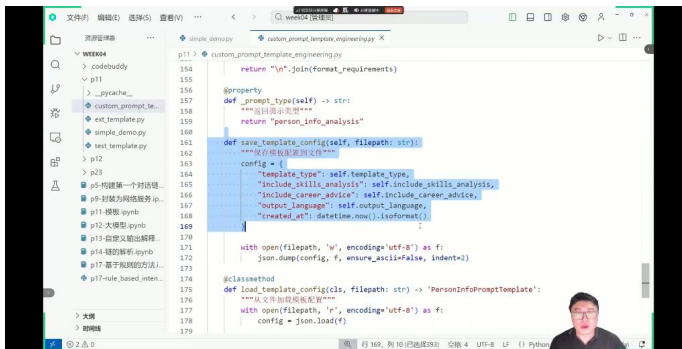
- 数据模型定义：
 - 使用Pydantic定义Person类，包含姓名(name)、年龄(age)、职业(occupation)等字段
 - 通过@validator装饰器实现年龄验证（0-150岁）
- 模板类结构：
 - 继承StringPromptTemplate基类
 - 支持模板分类参数(template_type)，默认值为"basic"
 - 必须传入person_info参数包含人员详细信息

2) 动态模板组合



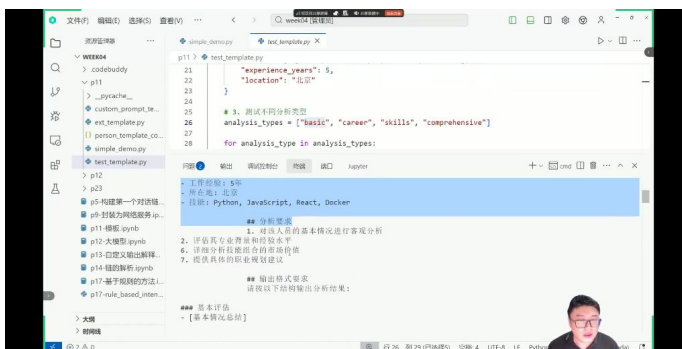
- 条件组合逻辑：
 - 基础分析要求包含：1.基本情况分析 2.专业背景评估
 - 职业发展类型(career)增加：3.职业轨迹分析 4.发展建议
 - 技能类型(skills)增加：3.技能结构分析 4.技能缺口识别
 - 综合类型(comprehensive)增加：3-5项深度分析
- 可选功能扩展：
 - 包含技能分析(include_skills_analysis)时追加第6项

- 包含职业建议(include_career_advice)时追加第7项
- ### 3) 工程化扩展功能



- 配置管理:
 - save_template_config方法将模板配置保存为JSON文件
 - load_template_config类方法从文件加载配置重建模板实例
- 属性装饰器:
 - 使用@property将方法转为属性, 简化调用接口
 - prompt_type属性返回固定模板类型标识"person_info_analysis"

2. 应用示例



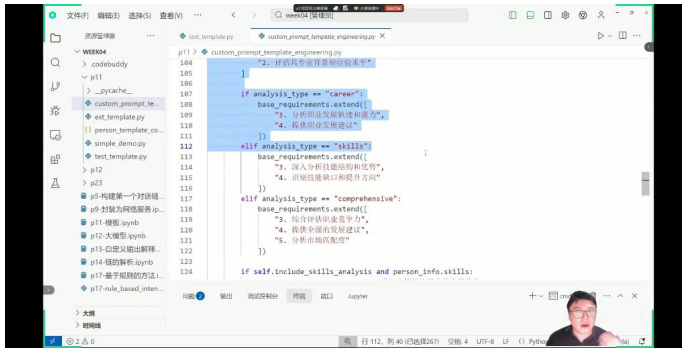
- 测试数据:
 - 姓名: 张三, 年龄: 28, 职业: 软件工程师
 - 技能: ["Python","JavaScript","React","Docke"]
 - 工作经验: 5年, 所在地: 北京
- 输出结构:
- 动态组合效果:
 - basic类型输出1-2基础项+6-7扩展项
 - career类型输出1-4核心项+可选扩展项
 - 通过analysis_type参数控制分析深度

三、优势分析 01:41:02

1. 模板的保存和读取 01:41:29

- 实现位置: 关键实现放在custom模块中, 业务代码使用est扩展方式
- 严格性: est检查机制更严格, 包含try/accept异常处理
- 格式要求: 比simple demo更严格的格式规范
- 兼容性: 模板保存读取等高级特性与custom完全一致

2. 模板的四种类型 01:43:40

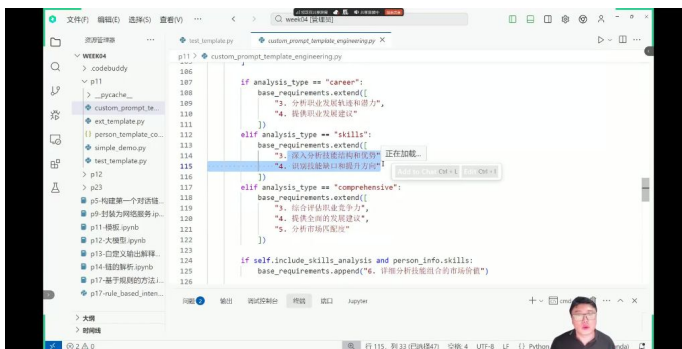


- 基础型(basic): 包含["1.基本情况分析", "2.专业背景评估"]
- 职业型(career): 扩展["3.职业发展轨迹", "4.职业建议"]
- 技能型(skills): 扩展["3.技能结构分析", "4.技能缺口识别"]
- 综合型(comprehensive): 扩展["3.竞争力评估", "4.全面建议", "5.市场匹配度"]

3. 模板与字符串拼接的关系 01:45:17

- 本质区别: 模板不仅是字符串拼接, 更是通过关键词驱动内容组合
- 工作逻辑: 通过format方法将参数传入template实例 (如person_info_prompt_template类)
- 循环应用: 使用for循环遍历预定义的4个关键词实现模板组合

4. 模板与prompt的逻辑配置 01:45:40



- 固定部分: 由模板控制 (如分析要求框架)
- 可变部分: 由prompt控制 (如具体分析类型参数)
- 条件扩展: 通过analysis_type判断进行动态需求扩展

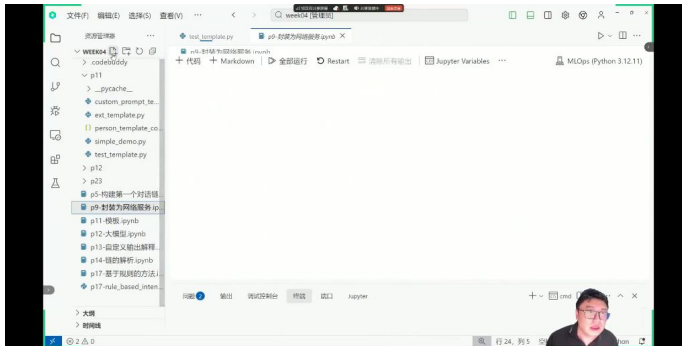
5. 模板的固定内容与prompt的可变内容 01:46:23

- 模板固定项: 职业/技能评估等基础分析项
- prompt控制项: 用户提交的具体信息 (姓名、工作经验等)
- 市场价值分析: 当include_skills_analysis=True且person_info.skills存在时自动追加

6. system prompt与user prompt 01:46:50

- system prompt: 提供回答环境和固定要求
- user prompt: 包含用户个人信息和特定请求
- 交互逻辑: system prompt验证user prompt信息完整性后生成回答

7. 测试数据与分析类型 01:47:15



- 测试数据：包含姓名、年龄、职业、技能等完整字段
- 分析类型：["basic","career","skills","comprehensive"]四种测试场景
- 执行流程：遍历测试类型并输出格式化分析报告

8. 模板与prompt的重新梳理 01:47:47

- 模板本质：system prompt的标准化结构
- 用户输入：通过user prompt传递个性化参数
- 格式控制：使用template.format()方法确保输入输出规范

9. 角色定义与提示词使用 01:48:12

- user角色：提交原始信息（姓名、职业等）
- system角色：提供分析框架和回答规范
- 协作模式：user prompt基于system prompt环境生成最终答案

10. system prompt与user prompt的逻辑关系 01:48:38

- 固定要求：如"姓名+工作情况"等必填字段
- 动态解答：根据用户提交信息生成定制化建议
- 验证机制：当信息不完整时可要求用户补充

11. 案例分析：固定要求与解答 01:49:33

- 案例结构：
- 设计要点：保持基础要求不变，动态扩展类型特定需求
- 错误处理：包含参数验证和异常处理逻辑

12. 用户信息的提交与解答 01:50:28

- 提交格式：支持自由格式（如"姓名:Wilson, 职业:程序员"）或结构化JSON
- 解析过程：通过大模型解析非结构化输入
- 响应生成：结合固定模板和个性化参数生成专业建议

四、发展建议 01:51:46

1. System Prompt与User Prompt的区别 01:52:01

- 功能定位：
 - system：提供回答框架和环境
 - user：传递具体请求参数
- 变更频率：system prompt相对稳定，user prompt每次交互变化
- 设计原则：system强调完备性，user注重灵活性

2. 数据解析与大模型应用 01:52:25

- 解析方式：直接使用预解析数据或通过大模型解析原始输入
- 数据验证：检查必填字段完整性和格式合规性

- **错误反馈**：对不符合要求的信息给出明确修改建议

3. 设计Prompt时的逻辑配置 01:52:36



- **模板控制项**：分析框架、输出格式等固定要素
- **prompt控制项**：具体分析类型、用户个性化信息
- **交互设计**：通过下拉菜单选择分析类型（职业/技能等）

4. 模板控制与功能变化的处理 01:53:05

- **多功能支持**：职业生涯规划、简历改写、转岗评审等场景
- **变化处理**：通过analysis_type参数区分不同功能需求
- **统一接口**：保持用户信息收集方式一致，动态调整分析逻辑

5. 用户选项与系统提示词的交互 01:54:14

- **选项设计**：将专业参数转化为用户友好选项
- **参数传递**：将用户选择映射为analysis_type等专业参数
- **提示词组装**：自动组合固定模板和用户选择生成完整prompt

6. 模板与函数调用的模拟 01:55:54

- **实现原理**：通过字符串模板模拟函数调用过程
- **参数传递**：使用类似template.format(param1=value1)的机制
- **扩展性**：支持基于条件的动态模板组合和内容扩展

五、技能分析 01:56:24

1. 用户信息分析框架

- **基础要求**：
 - 人员基本情况分析
 - 专业背景评估
 - 经验水平评估
- **扩展要求**：
 - 职业发展轨迹分析
 - 潜力评估
 - 职业发展建议

2. 模板工程体系 01:58:26

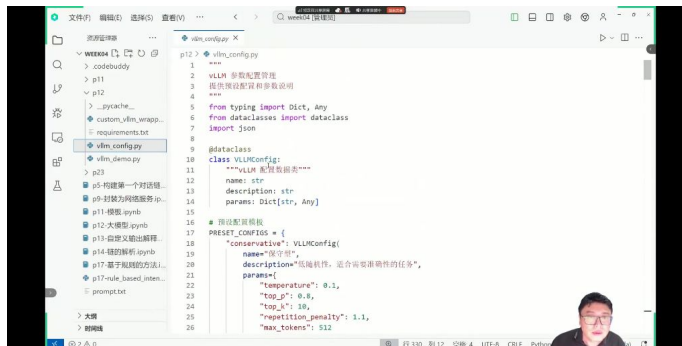
- **核心组件**：
 - 静态内容复用
 - 动态变量修改
- **技术要点**：
 - 字符串拼接技术
 - Python模板嵌套

- 前端模板语言整合
- ### 3. 大模型接口开发 02:00:06

1) vLLM自定义封装器 02:00:54

- **核心参数:**
 - **temperature:** 控制生成随机性 (0-2范围)
 - **top_p:** 核采样参数 (0-1范围)
 - **max_tokens:** 最大生成token数 (1-4096)
- **高级参数:**
 - **repetition_penalty:** 重复惩罚系数 (0.1-2.0)
 - **frequency_penalty:** 频率惩罚系数 (-2.0-2.0)

2) VLLM配置管理 02:02:59



- **配置方案:**
 - **保守型:** temperature=0.1, 适合准确性任务
 - **平衡型:** 通用场景默认配置
 - **创意型:** 高随机性配置
- **验证机制:**
 - 参数范围校验
 - 配置有效性检查

3) CustomVLLMWrapper封装 02:03:35

- **关键方法:**
 - **invoke():** 同步调用接口
 - **ainvoke():** 异步调用接口
- **特殊功能:**
 - 流式输出支持
 - 异常处理机制
 - 请求超时控制

4) 链式处理案例 02:06:27

● 项目报告数据模型

```
1 class ProjectReport(BaseModel):
2     project_name: str = Field(description="项目名称")
3     progress: str = Field(description="进度状态")
4     completed_tasks: List[str] = Field(description="已完成任务")
5     pending_tasks: List[str] = Field(description="待完成任务")
6     risks: List[str] = Field(description="风险点")
```

- **自定义输出解析器 02:12:09**
 - **处理流程:**
 - 接收原始文本输入

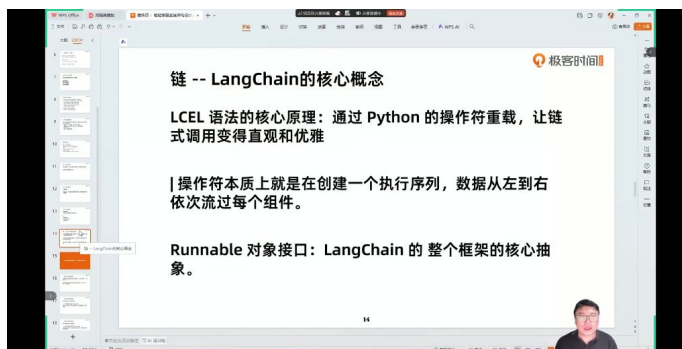
- 提取JSON格式内容
- 转换为结构化数据
- 关键技术:
 - 正则表达式匹配
 - JSON反序列化
 - 数据验证
- 链式调用演示 02:13:20

```

1
2 ##### 4. 创建处理链
3
4 chain = (
5     ProjectReportTemplate(language_style="professional")
6     | Tongyi(model_name="qwen-max")
7     | ProjectParser()
8 )
9
10
11 ##### 5. 执行处理
12
13 result = chain.invoke("移动应用开发项目，进度60%...")

```

6. LCEL核心原理



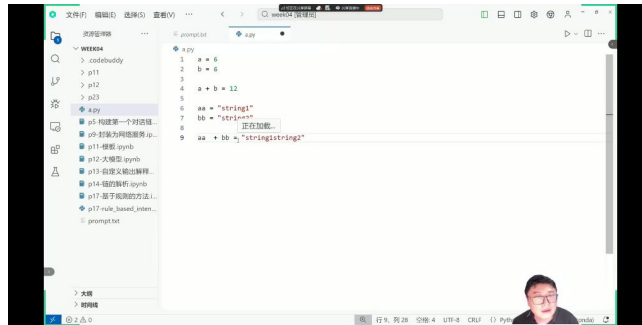
- 操作符重载:
 - | 运算符实现管道式调用
 - 数据从左向右流动
- Runnable接口:
 - 统一组件调用规范
 - 支持同步/异步执行
 - 实现错误处理和重试机制

六、链式调用的核心概念 02:18:23

1. LCEL语法 02:18:44

1) Python魔术方法基础 02:21:09

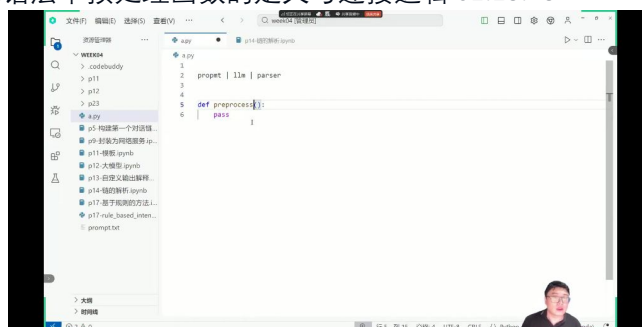
- 例题2: 字典的加法操作 02:22:42



- 运算符重载需求：当需要对字典实现key相连、value相加的特殊加法逻辑时（如 *dicta + dictb* 希望得到 *k1 + k2* 和 *3+5*），Python默认的加号无法识别这种自定义逻辑
- 接口预留机制：Python设计时通过魔术方法（如 `__add__`）为运算符预留了修改接口，允许开发者重载运算符功能
- 魔术方法：运算符重载 02:24:35
 - 底层实现：*a + b* 实际调用 `a.__add__(b)`，若 *a* 未实现则尝试 `b.__radd__(a)`
 - 方法特性：
 - 命名规范：以双下划线开头和结尾（如 `__add__`）
 - 继承机制：可通过类继承重写父类的魔术方法
 - 覆盖范围：包括算术运算、比较运算、下标访问等基础操作
 - 应用场景：需要改变对象默认行为时（如修改字典的方括号访问行为）

2) LCEL语法 02:26:05

- LCEL语法中|操作符与魔术方法的关系 02:26:08
 - 符号映射：管道符|对应Python的 `__or__` 魔术方法，*a|b* 等价于 `a.__or__(b)`
 - 反向调用：当左侧对象未实现 `__or__` 时，会尝试调用右侧对象的 `__ror__` 方法
- LangChain中Runnable对象对魔术方法的实现 02:26:21
 - 框架设计：所有Runnable对象（prompt/model/parser）都实现了 `__or__` 方法
 - 消息传递：重写后的方法会将 *a* 的 base message 传递给 *b* 的 base message
 - 连接本质：实质是将多个组件按顺序连接形成处理流水线
- LCEL语法中执行顺序与自定义逻辑的处理 02:27:00
 - 执行方向：始终从左到右执行（*a*→*b*），即使通过 `__ror__` 反向查找也保持该顺序
 - 开发便利：自定义组件只需关注核心逻辑，无需强制实现 `__or__` 方法
- LCEL语法中预处理函数的定义与连接逻辑 02:28:18



- 典型场景：在 `prompt|llm|parser` 前添加 `preprocess()` 函数
- 连接机制：当预处理函数未实现 `__or__` 时，LangChain会自动通过后续组件的 `__ror__` 完成连接
- 实现示例：`process | prompt` 实际执行 `prompt.__ror__(process)`
- LCEL语法中执行顺序与魔术方法的功能扩展 02:30:03
 - 框架演进：通过不断增加魔术方法的功能实现语法增强
 - 版本迭代：0.3版本后运算符重载的实现趋于稳定

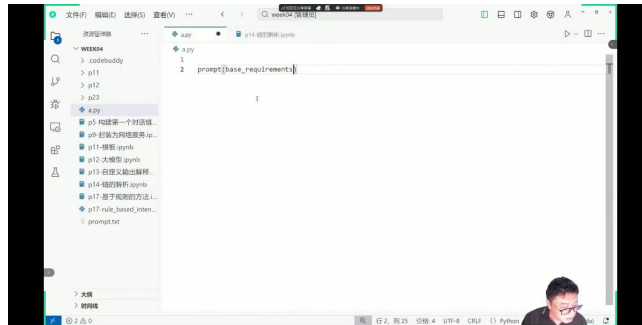
- **消息规范**：所有组件间传递的数据必须符合base message格式要求

3) Runnable对象接口 02:31:44

● Runnable对象概述 02:31:49

- **核心地位**：所有可运行工作单元的基础类，包含调用、异步、批处理、流式、并行等核心功能
- **常见子类**：Prompt（提示模板）、Model（大语言模型）、Parser（解析器）都继承自BaseRunnable
- **统一接口**：通过实现_or_和_ror_魔术方法提供链式操作能力

● Runnable对象的继承关系 02:32:03



- **三层架构**：
 - **基类**：BaseRunnable（定义核心接口）
 - **中间层**：BaseTemplate（提示模板）、BaseLanguageModel（语言模型）、BaseOutputParser（输出解析器）
 - **实现层**：具体Prompt/Model/Parser实现类

- **方法继承**：所有子类自动获得|操作符功能，通过继承父类的_or_和_ror_方法

● Runnable对象中的魔术方法 02:34:46

- **操作符重载**：
 - `a | b` 实际调用 `a._or_(b)`
 - 若`a`无_or_则尝试 `b._ror_(a)`
- **实现原理**：

● Runnable对象连接成可执行序列的原理 02:35:19

- **链式构建**：每次|操作生成新的RunnableSequence对象
- **嵌套结构**：
- **反向连接**：当左侧对象未实现_or_时，通过_ror_实现反向连接（预处理场景）

● Runnable对象序列的执行过程 02:37:37

- **执行流程**：
 - 执行第一个组件的invoke方法
 - 将结果作为输入传递给下一个组件
 - 依次传递直到最后一个组件
- **伪代码示例**：

● 工作流的执行与节点调用 02:38:50

- **节点关系**：每个工作流节点都是独立的Runnable对象
- **执行机制**：通过RunnableSequence自动管理节点间的输入输出传递
- **扩展性**：支持任意长度的工作流链式组合

● Runnable对象ror的执行过程 02:40:47

- **反向执行**：`a._ror_(b)`会先执行b再执行a
- **典型场景**：自定义预处理节点无需实现_or_方法
- **优势**：避免开发者实现魔术方法时的错误

● Runnable对象的核心功能与执行方式 02:41:41

- 同步执行: `runnable.invoke(input)`
- 异步执行: `await runnable.ainvoke(input)`
 - 无异步实现时会自动降级为多线程
- 批处理: 支持批量输入的并行处理
- 流式处理: 支持逐步输出处理结果
- Runnable对象的异步执行与批处理 02:42:43
 - 降级策略: 当异步方法未实现时自动转为多线程执行
 - 性能优化: 内部实现自动处理并发和资源管理
 - 统一接口: 同步/异步使用相同的方法签名 (`invoke/ainvoke`)
- Runnable对象的input与output 02:43:21
 - 类型约束:
 - `input_schema`: 定义输入数据类型
 - `output_schema`: 定义输出数据类型
 - 类型检查: 自动验证输入输出是否符合schema定义
 - 文档生成: schema信息可用于自动生成API文档

七、schema用法举例

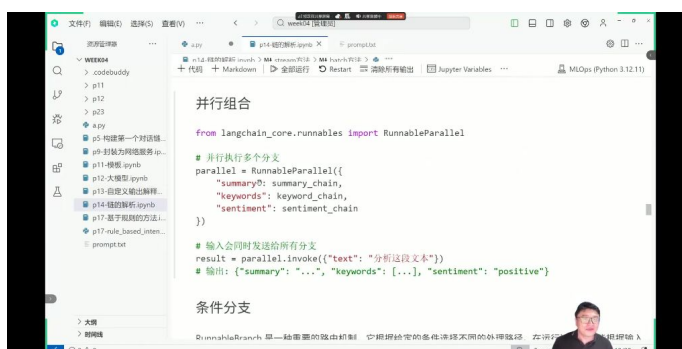
- 输入验证: 通过input schema检查必填字段 (如name和age) 和数据类型 (string)
- 输出规范: output schema固定包含AI message格式 (如system message)
- 应用场景:
 - API文档自动生成
 - 字段必填验证 (required标记)
 - 类型安全检查 (如string类型校验)
 - LangGraph组合监控 (探针机制)

八、链式调用方法 02:46:28

1. 基础调用方式

- 同步调用: 使用`invoke()`方法, 会阻塞直到返回结果
- 异步调用:
 - 使用`a_invoke()+await`实现非阻塞
 - 底层采用`asyncio`或线程池备用方案
- 流式调用:
 - `stream()`同步版本基于`yield`实现
 - `a_stream()`异步版本默认调用`async mock`

2. 批量处理方法

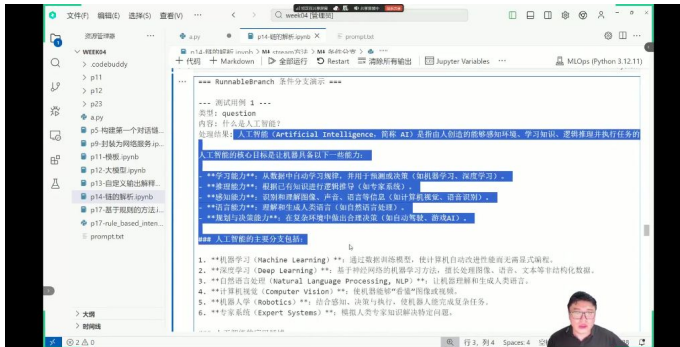


- 批量操作: 通过`batch()`方法处理列表输入 (如报告A/合同B/邮件C)
 - 执行特点: 自动并行化请求, 避免循环单独调用
- ### 3. 条件分支

1) 实现机制

- 路由判断：基于输入类型（如question/summary）选择不同链
- 类封装：通过RunnableBranch实现分支逻辑
- 层级差异：与基础链式调用属于不同抽象层级

2) 条件分支演示 03:05:15



- 类型判断：检测输入type字段（如question类型）
- 案例输出：
 - 学习能力：从数据自动学习规律
 - 推理能力：基于知识逻辑推导
 - 6大分支：包含机器学习/深度学习/NLP等

4. 并行组合

- 实现方式：使用RunnableParallel包装多个链（如summary/keyword/sentiment）
- 执行特点：输入同时发送给所有分支，输出合并字典
- 性能对比：相比串行执行显著提升效率

九、设计模式分析

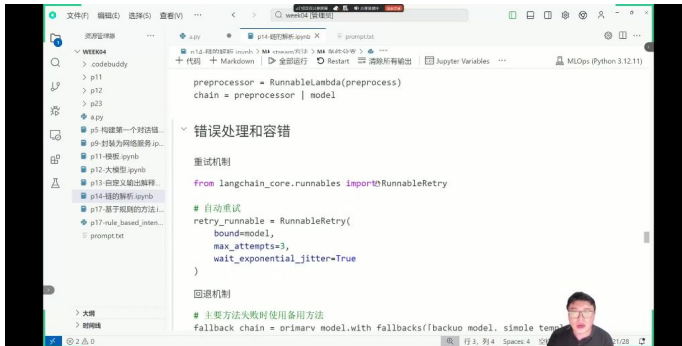
- 扩展局限：
 - 异步等特性通过外层包装实现
 - 复杂工作流表达困难（需改用LangGraph）
- 典型问题：
 - 分支嵌套时代码可读性差
 - 类型判断与业务逻辑耦合
- 优化方向：
 - 将路由/并行等逻辑抽离为独立节点
 - 采用DAG工作流替代链式结构

十、RunnableLambda 03:05:58

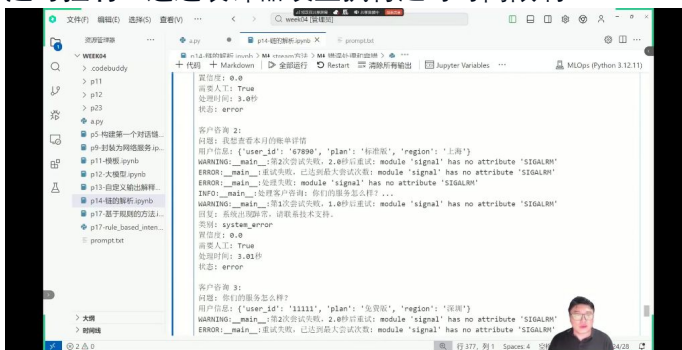
- 功能作用：将普通函数转换为Runnable对象，实现与大模型的兼容连接
- 调用机制：执行到Lambda位置时调用普通函数，例如预处理函数preprocess
- 典型应用：如将字符串转为大写的预处理函数preprocess，通过RunnableLambda(preprocess)转换后与模型连接
- 语法示例：preprocessor = RunnableLambda(preprocess); chain = preprocessor | model

十一、重试机制 03:07:01

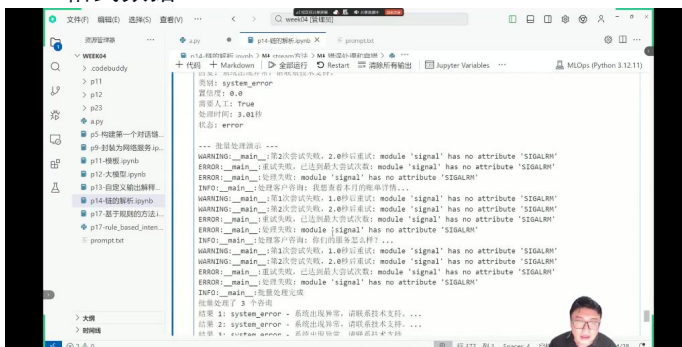
1. 错误处理和容错机制示例 03:07:23



-
- **核心组件**：RunnableRetry实现自动重试，需从langchain_core.runnables导入
- **参数配置**：
 - max_attempts=3：设置最大重试次数为3次
 - wait_exponential_jitter=True：启用指数退避策略（首次1秒，第二次2秒，第三次4秒）
- **回退机制**：主方法失败时使用with_fallbacks()调用备用方法
- **超时控制**：通过装饰器设置执行超时时间限制



-
- **错误场景**：
 - 信号模块缺失SIGALRM属性导致的系统错误
 - API调用出现500错误时的处理流程
- **处理流程**：
 - 首次失败后1秒重试
 - 二次失败后2秒重试
 - 三次失败后返回错误信息
- **输出格式**：包含错误类型、置信度(0.0)、人工处理标志(True)、处理时间(3.01秒)等JSON格式数据



-
- **批量处理**：同时处理多个用户咨询（如账单查询、服务咨询等）
- **错误特征**：
 - 相同错误在不同咨询中重复出现
 - 最终统一返回"system_error-系统出现异常，请联系技术支持"
- **监控指标**：记录每个咨询的处理状态、耗时和最终结果

十二、配置和定制 03:09:40

1. 传递配置参数 03:09:46

- **参数传递**：可通过config参数传递运行时配置，如config = {"temperature": 0.7, "max_tokens": 100}
- **默认配置绑定**：使用bind()方法绑定默认配置，如bound_model = model.bind(temperature=0.7)

2. 回退机制 03:13:50

- **三层容错设计**：
 - 第一层：主链处理，捕获异常并记录日志
 - 第二层：备用模型处理，组合primary_chain.first | backup_model | parser
 - 第三层：简单响应，返回预设错误信息
- **实现要点**：
 - 使用try-except嵌套结构
 - 每层失败后记录不同级别日志(WARNING/ERROR)
 - 最终返回结构化错误响应，包含response/category等字段

3. 标签和元数据 03:14:31

- **标签添加**：
 - 使用with_tags方法，如tagged_chain = chain.with_tags(["production", "v1.0"])
 - 作用：仅开发者可见，不影响业务数据，用于全链路追踪
- **元数据处理**：
 - 使用with_metadata方法添加业务元数据
 - 区别：元数据会与业务数据混合，影响系统行为
 - **最佳实践**：
 - 优先使用标签而非修改元数据
 - 保持原始数据完整性
 - 避免排查困难

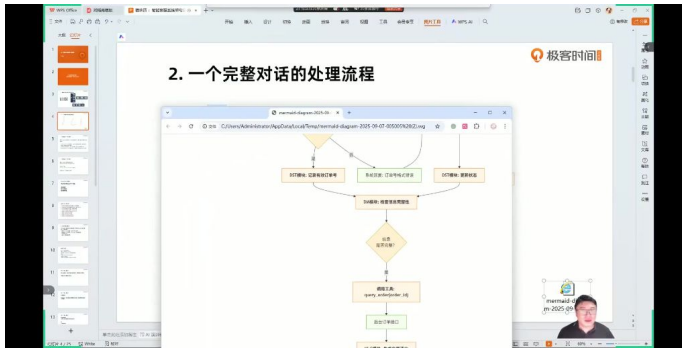
十三、链的解析 03:15:51

1. 任务留存 03:17:48

- **类型定义**：
 - 明确run方法参数类型：两个字典
 - 第一个字典：两个string参数
 - 第二个字典：string+any参数
- **并行使用**：
 - 避免滥用并行处理
 - **风险**：
 - 问题排查困难
 - 可能破坏依赖关系
- **错误标记**：
 - 通过日志记录错误状态
 - 典型字段：category/confidence/requires_human等

十四、智能客服系统构建设计 03:19:53

1. 一个完整对话的处理流程 03:20:51



- **核心模块：**
 - DST模块：处理订单状态记录与更新
 - DM模块：检查信息完整性
 - NLG模块：生成自然语言响应
- **异常处理：**
 - 多级重试机制（默认2次）
 - 响应时间监控（示例显示3.0秒）
 - 自动转人工条件：system_error且confidence=0

十五、知识小结

知识点	核心内容	重点/难点	应用场景
LongChain基础概念	处理链三要素：提示词模板、大语言模型、输出解析器	概念重新定义 （与常规理解不同）	对话系统开发框架
语言模型类型	分LLM（字符串输入输出）和ChatModel（消息列表输入输出）	角色类型区分（Human/AI/System等6种）	模型调用接口封装
提示词模板	支持f-string/jinja2/标准模板语法，支持多变量和条件逻辑	动态模板组合 （基础模板+业务逻辑模板）	多场景对话系统
输出解析器	支持JSON/XML/列表等格式转换，含类型验证和错误重试	链式处理中的格式转换枢纽	API数据标准化
处理链机制	竖线操作符实现组件连接，本质是运算符重载（ or ）	执行顺序控制 （从左到右严格串行）	工作流编排
异步处理	通过aiohttp实现异步调用，支持流式响应和批量处理	网络IO密集型场景优化	高并发服务
错误处理	指数退避重试机制（最大3次），支持fallback回退	生产环境 容错设计 关键	服务稳定性保障
监控体系	通过schema定义输入输出结构，支持metadata打标	全链路追踪实现基础	运维监控

性能优化	并行执行链 (RunnableParallel) 和条件分支 (RunnableBranch)	工作流复杂度控制	复杂业务逻辑处理
生产部署	FastAPI封装服务, 环境变量管理 (dotenv) , 模型 版本控制	密钥安全管理要点	企业级服务发布